Perch: Bounded Grants for Apps and Agents on Soroban Smart Accounts

Willem Wyndham and Elizabeth Engelman

The Aha Company

Abstract. Soroban smart accounts can let an application or agent act without a signature prompt per call, but the bound on that authority must hold across nested calls, repeated calls over time, and rule selection by whoever submits the transaction. We present Perch, a smart account in which the owner installs grants: rules that fix a target contract, functions, argument conditions, a signer threshold, an optional expiry and an optional sliding-window spending cap. The account decides ordinary authorizations at one point, with a pure function over the selected rules and their account-held spending records, linked into the contract. We translate this Rust function into Lean with Charon and Aeneas and prove that the translation agrees with a model on every input, that acceptance implies permission, and that attenuation, which the account cannot yet install, narrows authority. From empty records, each fixed capped rule's accepted charges stay within its limit over every window of positive ledgers; scoped lowering excludes the account and the supplied recovery contracts. These are properties of the translation under stated hypotheses; recovery completion follows a separate controller path. The account code around it is tested, not proved, and unaudited. We run Perch and an OpenZeppelin account with oz-policy-builder side by side in soroban-sim on one task, and compare the two designs.

Keywords: Smart accounts · Authorization · Delegation · Soroban · Formal verification · Lean

1 Introduction

An app that spends for a user needs authority beyond a single approval prompt. Session keys and capability systems already offer bounded delegation (§11); the question here is how a Soroban account enforces that bound. A Soroban account on Stellar can be a contract that decides each authorization itself (§2). A *grant* lets an app or agent act for the account within limits the account itself enforces. The bound has to hold across the nested calls of one transaction, across repeated calls over time, and whichever installed rule the transaction's submitter selects.

Our running example, the shop grant (Fig. 1), lets an app or agent send XLM from the account to the shop and to no one else, at most 10 XLM in any 100 ledgers (about eight minutes). A grant cannot say everything: it has no bound on one call's amount (the cap bounds the window), no *or* and no *not*, no reads of other contracts, and it checks each call alone.

(a) The grant document

```
{ "version": 1,
  "network": "Standalone Network ; February 2017",
  "signers": [
    { "id": "app", "verifier": "C...ED25519",
      "key": "...32-byte Ed25519 key, hex..." } ],
  "rules": [
    { "name": "pay-shop",
      "scope": { "type": "contract",
                 "address": "C...XLM" },
      "principals": { "type": "all",
                      "signers": ["app"] },
      "functions": ["transfer"],
      "args": [
        { "index": 0,
          "pred": { "type": "is-self" } },
        { "index": 1,
          "pred": { "type": "address-eq",
                    "address": "G...SHOP" } } ],
      "cap": { "limit": "100000000",
               "period-ledgers": 100 } } ] }
```

(b) In words

```
① The app signs with its own Ed25519 key;
  a verifier contract checks the signature.
② One target contract: the native XLM
① token.
③ Every listed signer must sign: here,
  the app alone.
④ Only transfer; any other function
② on XLM is refused.
⑤ Argument 0 is this account and
③ argument 1 is the shop: money leaves
  this account, and only to the shop.
④ ⑥ At most 10 XLM (100,000,000 stroops) in
⑤ any 100 ledgers, about eight minutes.

Nothing else: no other token, recipient,
function or signer, and no call to the
account itself.
⑥
```

Fig. 1: The running example: an app may pay the shop at most 10 XLM in any 100 ledgers (about eight minutes). (a) The grant document as the simulation installs it, with addresses and the key abbreviated: `C...` is a contract, `G...` an account controlled by a key pair. (b) Each marked field in words.

We present Perch, a Soroban smart account that offers safe, bounded delegation for apps and agents: simple, proved, and testable end to end in soroban-sim. *Safe* and *bounded* refer to three core properties under their stated hypotheses: each accepted context is allowed by a supplied rule; from empty records, each fixed capped rule bounds its accepted charges; and scoped lowering excludes the account and the supplied recovery contracts (Theorems 2, 6 and 8). *Simple* means one document, one install call (`apply_grant`) and one decision point, at the cost of fewer conditions. *Proved* applies to the Lean translation of the Rust decision function, not the deployed Wasm or the tested account code around it, the *shell*. *End to end* covers both designs' contract Wasm and the token in one local host, not the wallet, RPC, passkeys or fees. A completed recovery moves one counter, and every earlier grant stops authorizing.

Status. The design is merged into Perch's `dev` branch, not yet `main`; an earlier build is previewed on testnet through Nido [18], and not audited; we do not recommend it for real funds before an audit. An earlier design is on mainnet [21]. soroban-sim [25] is published by The Aha Company, where the authors work.

Revision boundary. The claims and recorded artifacts describe the pinned revisions in Appendix A.

Contributions. (1) An account design with one decision point and account-held spending windows (§§3–4); (2) Lean theorems about the translation of its decision function (§8); (3) a dual soroban-sim simulation of Perch and an OpenZeppelin account with oz-policy-builder from the same contract bytes (§7); and

(4) a source-cited comparison, including where oz-policy-builder is ahead (§6). We ask whether the account enforces grants and caps as designed (**RQ1**, §§7 and 9), what is proved, tested or trusted (**RQ2**, §8), and how the designs differ on one task (**RQ3**, §§6–7). Companions cover the comparison, *Bounding an App's Authority on Soroban: Perch and oz-policy-builder Compared*[1]; all theorems, *What Perch Proves: An Aeneas-Translated Authorization Evaluator and Its Models*[2]; and recovery and agents, *The Perch Vision: Recovery, Agents and the Full Account*[3].

2 Background

Soroban authorization. A contract that needs an address's approval calls `require_auth`; for a contract account, the host then calls the account's `__check_auth` [13,26] with the authorization entry's signed payload (network ID, nonce, expiration ledger, invocation tree), its signatures and its *contexts*: every call and contract creation in the tree, in preorder, each of which the account must decide. A lost guess in a game gives two, the game's `guess` and the `transfer` it makes from the account. The nonce is consumed only on success, so a refused entry can be resubmitted until it expires. CAP-0071 [14] lets a delegated signer's approval ride in the account's own entry. Recording-mode simulation, which builds entries, does not call `__check_auth`, so only enforcing simulation predicts a refusal.

OpenZeppelin and oz-policy-builder. OpenZeppelin's Soroban library [20] stores *context rules* with *external* signers (a verifier contract and key), *delegated* signers (addresses) and *policies*, contracts with an `enforce` hook; the payload names one rule per context (`context_rule_ids`). In v0.8.0, `__check_auth` validates the selected rules, authenticates the signers, then calls each policy's `enforce` (`storage.rs:629-690`). oz-policy-builder [28] has two halves (`README.md:13-24`): off chain, a synthesiser turns a recorded transaction or a declared intent into a predicate, verified against the deployed interpreter's `check`; on chain, a shared `policy-interpreter` policy evaluates it on each guarded call alone and keeps no totals, which OpenZeppelin's `spending_limit` policy keeps (`docs/stellar.md:404-407`). Perch reuses OpenZeppelin's types and payload field; it differs in what decides.

3 Grants and Spending Windows

Installing a grant. The owner calls `apply_grant` with a grant document (Fig. 1a). A stateless document compiler validates the JSON; the account lowers it with

[1] https://bitswell-hq.github.io/stellar-registry__perch/perch-paper/oz-policy-builder/

[2] https://bitswell-hq.github.io/stellar-registry__perch/perch-paper/verification/

[3] https://bitswell-hq.github.io/stellar-registry__perch/perch-paper/vision/

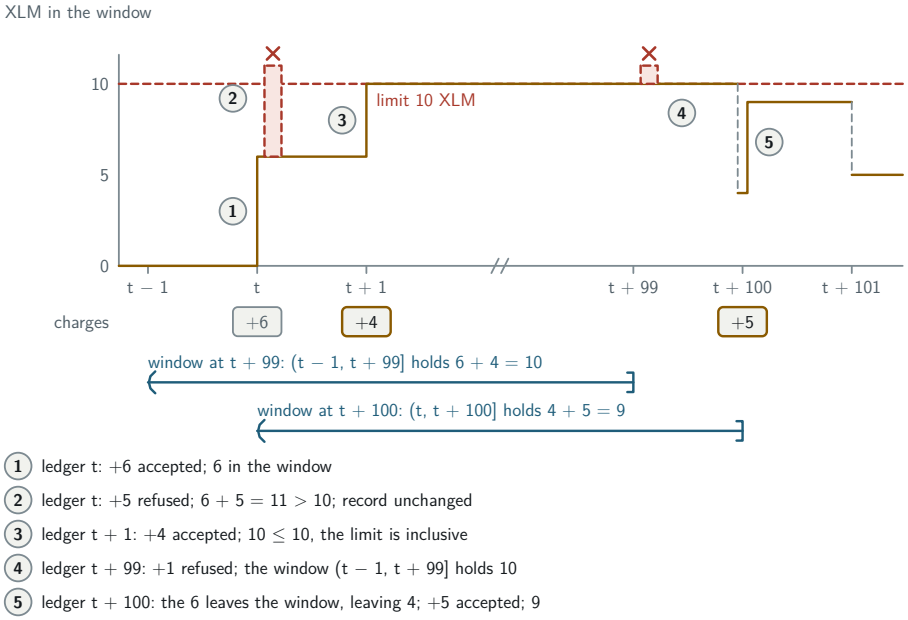


Fig. 2: The shop grant's cap, 10 XLM per 100 ledgers, on illustrative ledgers. At ledger L the window holds charges made after $L - 100$ and through L. Charges inside the window at $t + 100$ are highlighted. A refused transfer leaves the spending record unchanged. A charge stops counting exactly 100 ledgers after it was made.

rule-core's `lower_scoped` under its own address, its *excluded set* (its adopted recovery controller, and the zero-knowledge adapter and pool of its recovery configuration) and its grant generation, refusing any rule that targets the account or that set (Theorem 8).

What a rule checks. One target contract and a function list; five kinds of argument condition (`is-self`, `address-eq`, `string-in`, `string-prefix`, `u32-eq`), all of which must hold; a threshold over distinct listed signers; optionally an expiry (`not-after-ledger` N authorizes through ledger $N - 1$); and optionally a cap, when the function list is exactly `["transfer"]`. The payload names one rule per context, the account does not search, and a signer that no chosen rule names is refused.

The spending window. For a cap of λ per P ledgers, the window at ledger L holds the charges made after $L - P$ and through L (Fig. 2). The decision that checks a call also charges it, contexts of one authorization that select the same rule see each other's charges, and a refusal changes nothing. The record lives in the account's storage and holds up to 907 spends; replacing a capped rule starts an empty window. Theorem 6 bounds every window from empty records.

One authorization through `__check_auth` at the pinned build, in six stages.

No.	Step	Exit and code	Assurance
1	**Host call.** The Soroban host calls `__check_auth(signature_payload, auth payload, contexts)`; the auth payload names one rule per context (`context_rule_ids`) and the claimed signers.	–	trusted (host)
2	**Guards.** Refuse a context calling a reserved hook name (`install`, `uninstall`, `enforce` and seven `rcv_*` names), an account frozen during a recovery (the vision paper), and a selected grant rule on the account's adopted recovery contracts. A recovery completion leaves here.	`ReservedFunction` (1); `AccountFrozen` (2); `GrantEscalation` (4); a completion: decided by the recovery controller	tested
3	**Select rules.** Resolve each named rule ID to a stored rule and its spending record; the account does not search.	`ContextRuleNotFound` (3000)	tested
4	**Decide.** `rule_eval::check_auth` maps the rules, records, contexts, choices, claimed signers, ledger and generation to the next records or a refusal. It reads no storage and checks no signature.	stale grant: `StaleGrant` (3); otherwise `RuleEvalRefused` (5)	proved
5	**Authenticate.** Hash the payload, the account and the rule IDs into a digest; each external signer's verifier contract checks it (trusted); a delegated signer authorizes through the host.	bad signature: the verifier traps (`Error(Crypto, InvalidInput)` for Ed25519); `ExternalVerificationFailed` (3003)	tested; verifier trusted
6	**Store.** Store each changed spending record. Authorized.	–	tested

No exit writes anything; the verifier runs only after the decision accepts.

Fig. 3: One authorization in six stages. Stage 4 (decide) is the function the theorems cover; the other stages are tested account code, and the host is trusted. The vision companion paper splits these six stages into nine steps.

Attenuation (modelled; not installable). A grant derived for a helper by rule-core's 10 attenuation steps allows no call its parent does not (Theorem 4); caps and expiry narrow by trace inclusion (Theorem 5 of the verification companion), and neither bounds what parent and child spend together. Installing one is proposed.

Ending a grant. A grant stops at its expiry, by one owner call (`remove_grant`, which deletes its storage), or, with every earlier grant, when a recovery completes; anyone may then remove a stale grant's storage with `gc_grant`, which refuses a current-generation grant.

4 The Authorization Path

Figure 3 follows one authorization through six stages: the account decides at one point, writes nothing on a refusal, and calls the verifier only after the decision accepts.

The decision function. Stage 4 is rule-core's `rule_eval::check_auth` [29] (`rule-eval/src/authorize.rs:239`):

Table 1: What a grantee or a relayer can do, and what holds. T2, T6 and T8 are Theorems 2, 6 and 8 of the formal results; O-labels name the obligations on the account code around the decision function, which the verification companion paper lists. The vision companion paper covers owner-key theft, guardians and the verifier registry.

Who	Can do	What holds (evidence)	What remains
A1 Grantee (app or agent key)	Sign within rules naming its key; build any invocation tree; choose among rules naming its key	T2: every call of an accepted request is allowed by an installed rule (signer authentication, selection and context conversion tested, O1, O2, O7). T6: each capped rule's window stays within its limit (selection and record persistence tested, O2, O6). T8: no grant rule allows a call to the account or its adopted recovery contracts (proved for lowering; the runtime recheck tested)	The union of authority across rules naming its key; caps per rule, not per grant; each call checked alone; allowances and `execute`; `approve` through a token rule with no function list; replacing a rule starts a new window
A2 Submitter or relayer	Wrap signed entries, choose timing and fee, resubmit a refused entry until its expiration ledger	An external signer's signature binds this account, the entry and the selected rule IDs (tested; a paper proof under signature and hash assumptions)	Delegated signers do not bind rule IDs. A rule edited in place between signing and inclusion decides under its new contents

```
pub fn check_auth(rules: &[Rule], records: &[SpendRecord], contexts: &[Context],
    choices: &[u32], authenticated: &[Signer], now: u32, generation: u64)
    -> Result<Vec<SpendRecord>, Refusal>
```

It returns the next spending records or a `Refusal`, reads no storage and checks no signature (Definition 3 is its model). The account reports its refusals as auth error 3 (a stale grant) or 5 (any other), inside `Error(Auth, InvalidAction)`, beside the other stages' exits in Fig. 3; a wallet that needs the reason re-runs the function off chain.

What a signature binds. An external signer signs a digest of the host's payload, this account and the selected rule IDs. A delegated signer (CAP-0071) approves the host's payload, which omits the rule IDs, so a submitter may select any rule that lists the delegate; OpenZeppelin v0.8.0 binds them for both kinds.

5 Threat Model

A grant has to hold against the app that holds it and against whoever submits its transactions: A1, the grantee's key, honest or stolen, and A2, a submitter or relayer. Table 1 gives what each can do, what holds and what remains. In Saltzer and Schroeder's terms [23], `__check_auth` mediates every `require_auth` on the account that an authorization entry satisfies; one satisfied by invoker authority, as in the owner's `execute`, never reaches it.

Goals and trust. Containment (G1), permission soundness (G2) and window safety (G3) are proved for lowering and the decision function, the code around them tested; fail-closed refusal (G6) is tested; recovery invalidation (G4) is

proved in an account model (Theorem 10 of the verification companion), tested and run on testnet. Other goals and adversaries are in the vision companion. We trust the Soroban host and validators, the verifiers documents name, the adopted recovery controller, the document compiler, rustc and the wallet's display, and assume unforgeable signature schemes (Ed25519; ECDSA P-256 for passkeys) and a collision-resistant SHA-256, under which the verification companion proves on paper that an external signature binds the account, the entry and the rule IDs. Budgets across grants, effects outside authorization, confidentiality and verifier correctness are non-goals.

6 Perch and oz-policy-builder

Shared ground, two arrangements. We cite oz-policy-builder at 43b13ed from its public documentation and source. Both designs build on OpenZeppelin's context rules, judge each call alone (`docs/architecture.md:307-309`) and deny by default. OpenZeppelin with oz-policy-builder composes a decision from contracts attached to a rule (interpreter, `spending_limit`, a threshold policy), each swappable and auditable on its own. Perch puts one document behind one decision point that holds every check and its state. Composition buys flexibility, such as a check that reads a live price, which a Perch rule cannot express; one decision point gives one place to check and to keep a limit's state.

A limit over time. The interpreter "does not see what a transaction moved, keep rolling totals or count calls" (`docs/stellar.md:404-407`); totals come from `spending_limit`, an opt-in policy that accepts `transfer` alone (`policy-synth/src/install.ts:957-967`). A Perch window is part of the rule's decision (Theorem 6).

Where each is stronger. oz-policy-builder is ahead in *maturity* (Table 2, first three rows), *authoring* (record or declare, synthesise a predicate, test that each conjunct is needed, and verify it) and *expressiveness and ecosystem* (reads of other contracts, quantifiers, clock windows, batches related through an execution adapter, threshold policies, a deny code per reason, and rules on existing OpenZeppelin accounts, where a grant needs a Perch account). Perch integrates the *window cap* into *one decision point*: evaluator refusals return before authentication. It installs one grant across contracts and supplies *proofs about that core*. Both designs enforce the configured window in our task; removing OpenZeppelin's policy demonstrates the effect of that configuration change, not a failure of its cap. No formal model was found in oz-policy-builder's tree, which is tested, partly on the shipped Wasm.

Lessons for Perch. We propose five: a front end that records a transaction and emits a grant document for review; simulating a grant against the account's own decision function before install; minimality checks, and overlap checks that refuse a cap another rule naming the same key bypasses; a corpus of real transactions; and an audit before any claim of maturity.

Table 2: oz-policy-builder and Perch, side by side. Read from source at the pins; nothing here is measured. The full table, with a source for each cell, is in the oz-policy-builder companion paper.

	oz-policy-builder on an OpenZeppelin account (`43b13ed`)	**Perch (`0b3752d`)**
Status	Release v5.2.0 and later commits; interpreter pinned on testnet, none on mainnet (`config.ts:57-80`); installs on existing OpenZeppelin accounts (0.7.2-line and v0.8.0 builds, `policy-synth/README.md:143`)	On Perch's `dev` branch, not yet `main`; an earlier build as a testnet preview; earlier design on mainnet; a grant needs a Perch account
External review	Audit of v5.2.0 in progress since 26 October 2026; off-chain packages hardened after a penetration test (PR #78, `fix/hacken-pt-findings`, merged at `43b13ed`) (`README.md: 57-59, 166-171`)	None completed
Tested on real transactions	33 mainnet transactions of 26 shapes, including SEP-41 transfers, Soroswap swaps and Blend submits, taken from synthesis to the interpreter (`packages/policy-synth/src/corpus.test.ts:1-3`; `fixtures/corpus/mainnet.json`); scripted testnet runs of the executor binding, the adapter, a no-gate liquidation and the v0.8.0 migration (`evidence/README.md:18-24`)	11 transactions through Nido's testnet preview; local reference runs
Authoring	Record a transaction or state an Intent, synthesise a predicate and test that each conjunct is needed, verify it against the deployed `check`, review, unsigned install; nine tools over a CLI and an MCP server; three packages at 5.2.0 (`README.md:13-24`; `packages/*/package.json:3`)	Write a grant document (perch-js builder); the account's compiler validates it at install
Conditions on a call	And/or, comparisons, quantifiers over lists, ledger-clock windows (`until`/`from`), slippage floors (`minOut`), reads of other contracts (`dsl.rs:84-158`; `intent.ts:18-49`)	Target, functions, five conjunctive argument conditions, threshold, expiry; no reads
Limit over time	OpenZeppelin `spending_limit`, a separate opt-in policy on `transfer` (`docs/stellar.md:404-407`; `install.ts:957-967`)	A window cap inside the rule, charged in the same decision (T6)
Where it is decided	The account's checks, then each attached policy in stored order	One decision function in the account
Signers	Delegated signers on policed rules; thresholds through a threshold policy (`README.md:122-125`)	External (passkey, Ed25519) and delegated; threshold in the rule
What a refusal reports	A deny code per reason (`policy-interpreter/src/storage.rs: 27-86`)	Auth error 3 or 5
Evidence about the decision	Tests, partly on the shipped Wasm; a STRIDE threat register	Lean proofs about the evaluator's translation; the account code around it tested
Ending authority	Rules removed one at a time	One owner call per grant; a completed recovery stops every earlier grant from authorizing through the account

7 Both Designs in soroban-sim

soroban-sim [25] runs the contract code of both designs offline in one host; the HTML edition[4] embeds it for the reader to drive, and we report its recorded reference run.

[4] `https://bitswell-hq.github.io/stellar-registry__perch/perch-paper/sim/`

What runs. Each design gets a session on soroban-env-host 29.0.2 at protocol 29, applying with enforcing authorization in a recording-mode build: OpenZeppelin's v0.8.0 account with oz-policy-builder's `policy-interpreter` and `spending_limit`, or the Perch account with its document compiler and an Ed25519 verifier, each beside the native XLM contract, rebuilt from the pinned sources (Appendix A; only the OpenZeppelin side matches a deployment). The OpenZeppelin predicate pins the recipient and bounds each transfer at 10 XLM, with `spending_limit` adding 10 XLM per 100 ledgers; Perch runs the shop grant. Each request records its verdict and code, the contracts called during authorization (from the host's diagnostic events), the window and a ledger fingerprint. The OpenZeppelin side signs as a delegated account, since oz-policy-builder refuses external signers on policed rules (`docs/stellar.md:563-564`), hence 2 authorization entries to Perch's 1. We report no instruction counts.

The recorded scenario (Table 3). A 6-XLM payment applies in both designs. The 5 XLM that follows is refused in both: Perch's trace shows `__check_auth` refusing with no verifier call, the OpenZeppelin account's shows `spending_limit` refusing (3221). With `spending_limit` removed, the OpenZeppelin account applies each of three 6-XLM sends under the per-call bound (18 XLM), as oz-policy-builder documents, while Perch refuses all three because its window holds 6. Re-adding `spending_limit` starts an empty history, as replacing a Perch rule starts an empty window. Another recipient, a forged signature and a key not on the rule are refused by both; 100 ledgers later, 5 XLM applies in both.

Why soroban-sim. Real contract bytes run offline and deterministically, every verdict carries a trace, and native, WASI and browser runs return byte-identical replies, timings removed.

8 Formal Results

Theorems 2, 6 and 8 below are the three bounds of §1. Appendix B states Theorem 1 (on every input the translation returns the model's result, without panicking or diverging) and Theorem 4 (attenuation never widens the calls a grant allows; not installable). Theorem numbers are shared with the verification companion, which states all 13.

The evaluator and its translation. The decision function is about 2,500 lines of `no_std` Rust in rule-core [29]. Charon [10] reads its MIR (rustc nightly-2026-09-17) and Aeneas [11] translates it into Lean [17]; CI regenerates the translation, fails on any difference, and pins all 1,005 exported theorems to Lean's standard axioms. *Proved* means kernel-checked.

What the proofs trust. The Lean kernel and its standard axioms; Charon 7b462d9, Aeneas ab34bd2 and its Lean model of Rust primitives; rustc's MIR; and the specification, including the denotations. The account, as built for the pin and the testnet preview with rustc 1.97.1, also relies on the document compiler, the tested shell and verifiers, and the Soroban host.

Table 3: The recorded run, one command a row. Recorded by soroban-sim's native run of the simulation page. OpenZeppelin requests carried 2 authorization entries, Perch requests 1. The window is the Perch grant's record, or `spending_limit`'s history (none while it is removed). Every refusal was an enforcing simulation that left the ledger unchanged. Payments go to the shop unless noted. The window column is the stored record; spends 100 or more ledgers old leave it at the next charge.

Step Request	OpenZeppelin account	Perch account
0 set up	set up; 100 XLM	set up; 100 XLM
1 install the rule	applied	applied
2 pay 6 XLM	applied · window 6 of 10	applied · window 6 of 10
3 pay 5 XLM	refused, 3221 in `spending_limit` · window 6 of 10	refused, 5 in the account · window 6 of 10
4 `spending_limit` off	removed	—
5 pay 6 XLM	applied · no window	refused, 5 in the account · window 6 of 10
6 pay 6 XLM	applied · no window	refused, 5 in the account · window 6 of 10
7 pay 6 XLM	applied · no window	refused, 5 in the account · window 6 of 10
8 `spending_limit` on	re-attached	—
9 pay 1 XLM	applied · window 1 of 10	applied · window 7 of 10
10 pay 1 XLM elsewhere	refused, 100 in the interpreter · window 1 of 10	refused, 5 in the account · window 7 of 10
11 pay 1 XLM, another key as the app	refused, Error(Crypto, InvalidInput) in the account · window 1 of 10	refused, Error(Crypto, InvalidInput) in the verifier · window 7 of 10
12 pay 1 XLM, key not on the rule	refused, 3016 in the account · window 1 of 10	refused, 5 in the account · window 7 of 10
13 close 100 ledgers	100 ledgers closed · window 0 of 10 (record lists 1 XLM)	100 ledgers closed · window 0 of 10 (record lists 7 XLM)
14 pay 5 XLM	applied · window 5 of 10	applied · window 5 of 10

Notation. Lists carry an arrow ($\vec{c}$); ρ is a rule set R's spending records, one per rule; $r_i = R_{j_i}$ is the rule chosen for context c_i; $u_r = \infty$ and $g_r = \bot$ when absent; $\varnothing$ is the empty records; ledgers are natural numbers. $\mathit{allows}(r, c, S)$: c calls r's contract and a function it allows, meets r's argument conditions, and S holds at least k_r of r's signers. *checkAuth* returns records ρ' when each chosen rule allows its context, has not expired and carries generation $\bot$ or g, every signer is on a chosen rule, and every cap charge passes. spent_j lists each positive charge (ledger, amount) that a run's accepted authorizations make to rule j, whose cap is (λ_j, P_j). *Derives*: zero or more of the ten attenuation steps. Appendix B gives Definitions 1–5.

Theorem 2 (Acceptance implies permission).

$$\mathit{checkAuth}(R, \rho, (\vec{c}, \vec{j}, S, \ell, g)) = \rho' \\ \implies \forall c \in \vec{c}.\ \exists r \in R.\ \mathit{allows}(r, c, S)$$

If evaluation accepts, the selected rule set allows every supplied context for the signer list it was given. The theorem takes that list as an input; the account authenticates it (obligation O1).

Hypotheses: The signer list is an input; the account authenticates it (O1)

Evidence: Chain.RuleSet.checkAuth_allowsAll and Extracted.check_auth_allows_all · PROVED · MODEL + TRANSLATION; the chosen rule as witness: RuleSet.checkAuth_chosen · PROVED · MODEL

Theorem 6 (Cap-window safety).

$$\begin{aligned} & \mathit{spent}_j(R, \varnothing, \vec{a}) = ((t_k, x_k))_{k=1}^{m} \\ & \quad \Longrightarrow \ \forall L. \sum_{k:\ \max(0,\, L-P_j)<t_k\le L} x_k \ \le\ \lambda_j \\ & \ \wedge\ \big((\forall a \in \vec{a}.\ \ell_a \ge 1) \implies \\ & \quad \forall s. \sum_{k:\ s\le t_k<s+P_j} x_k \ \le\ \lambda_j\big) \end{aligned}$$

From empty records, in any run, counting its accepted authorizations, the charges made to a capped rule with a non-negative limit after $L - P$ and through L sum to at most its limit, for every ledger L. The model also bounds every P consecutive ledgers that start at ledger 1 or later, and those from ledger 0 when no authorization is at ledger 0.

Hypotheses: One fixed rule set. Rule j of R has the cap (λ_j, P_j) with $\lambda_j \ge 0$; empty records. The second conjunct is proved for the model only. Without its hypothesis it fails from ledger 0: two charges of the limit at ledger 0 are both recorded (`Cap.period_ledger_zero`).

Evidence: RuleSet.window_le_limit; Extracted.runAuthorizations_window_le_limit · PROVED · MODEL + TRANSLATION. RuleSet.period_le_limit_of_pos, period_le_limit · PROVED · MODEL. From ledger 0: Cap.period_ledger_zero

Theorem 8 (Grant target containment).

$$\begin{aligned} & \mathit{lowerScoped}(d, A, X, g) = R \\ & \quad \wedge\ \mathit{Derives}(R, R') \\ & \quad \wedge\ \mathit{allows}(R', \mathsf{call}(C, f, \vec{v}), S) \\ & \qquad \Longrightarrow\ C \ne A\ \wedge\ C \notin X \end{aligned}$$

$\mathit{lowerScoped}(d, A, X, g)$ is the rule set the account installs for grant document d at address A, with excluded set X and generation g; it allows no call to A or to an element of X, and attenuation preserves this. In the account, X is its adopted recovery controller and the zero-knowledge adapter and pool of its recovery configuration.

Hypotheses: Lowering under address A, excluded set X and generation g; for the translation, each rule's argument conditions fit the evaluator's bounded vectors (`ArgsRoom`)

Evidence: Model: Document.Doc.lowerScoped_allows_excluded, Doc.lowerScoped_attenuateAll_excluded; translation: Extracted.lower_scoped_allows_excluded, Extracted.derives_allows_excluded · PROVED · MODEL + TRANSLATION

9 Evaluation

RQ1: enforcement. Perch's tests run 58 Lean-generated traces through the natively compiled account and check every verdict and spending record against the model. In the recorded run (§7) each Perch verdict is the one the shop grant specifies, and the designs agreed except on the three sends after the owner removed `spending_limit`.

Testnet preview, earlier build. On 5 November 2026, a scripted run of Nido's preview, on a deployment built from Perch `a410123` before the pin, with virtual passkeys made 11 testnet transactions, among them a game's two-rule grant, two lost guesses and a one-guardian recovery to completion; the wallet, not the chain, declined a third guess over the cap.

Cost, Perch only. On that earlier build, a passkey-signed transfer through a grant paid a testnet fee of 12,889 stroops, WebAuthn verification included; a fresh preview account holding the game's grant, after two capped transfers (3,308 bytes), pays 1.677 XLM a year in rent at the floor rate.

Threats to validity. One task; one host version, recording-mode, without exact metering; signer kinds that differ by necessity; testnet resets, with envelopes not yet archived; and a game run from a branch with an uncommitted patch.

10 Limitations

The theorems concern the translation, not the Wasm rustc builds; the specification is trusted; the shell is tested; and no single theorem covers the whole authorization path. Table 1 lists what a grantee or a relayer can still do. Beyond it, a record holds at most 907 spends; two pre-signed installs for one grant can land in either order; and document caps (10 rules, 8 signers, 8,192 canonical bytes) bound what a grant can say. Nothing is audited; the owner key, recovery and the trusted components are in the vision companion.

11 Related Work

Hardy's confused deputy [9] motivates designating authority explicitly, as capability systems do. SPKI/SDSI [7], macaroons [1], Biscuit [2] and UCAN [27] attenuate tokens a holder carries; a windowed cap needs past spending, which a token cannot carry without an external store, and in Schneider's terms [24] makes the decision a stateful execution monitor. On EVM chains, ERC-4337 [3] and ERC-7579 [32] validators and hooks play the role of OpenZeppelin policies, ERC-7710 [15] and ERC-7715 [12] carry delegations and permission requests, and Zodiac Roles [8] scopes a role to targets, functions and parameter conditions with refilling allowances. MetaMask's delegation framework [16] caps transfers per fixed period, resetting at each period's start, so spends either side

of a reset can reach twice the cap within one period's length; a Perch window $(L - P, L]$ is bounded for every L (Theorem 6). In XACML's terms [19], Perch links its decision point into the account, where an OpenZeppelin account splits it across its own checks and policies. Cedar [5] links a Lean model to its Rust by differential testing [6]; Perch proves a translation of its Rust equivalent to a model. oz-policy-builder's synthesis relates to policy mining [31]. Sunbeam [4] and Komet [22] reason about compiled Wasm, which complements a proof about a translation, and soroban-sim [25] runs that Wasm in a local host.

12 Future Work

Everything here is proposed: an agent runtime with delegation-tree budgets; an account entry point for attenuated grants; and Cedar as an authoring language. The vision companion details each.

13 Conclusion

Perch decides an app's grant at one point in the account and charges its spending window in that decision. The decision function's translation is proved to match a model and to keep the claim's three bounds; the shell is tested, and nothing is audited. In the recorded soroban-sim run, the two designs' verdicts differed only where the owner removed the limit over time. oz-policy-builder is ahead in maturity, authoring and expressiveness, and Perch should learn from it. What remains is to merge the stack, complete an audit, and measure both designs with exact metering.

A Artifacts

Pins. Perch `0b3752d8a179a5952b05fbebd29131dbbf99fa80` [30], with the earlier design on mainnet at `c135aca277745434f2abf679edd9868c95204b51`; rule-core `a3c92ba074dcac4052727b7599e5fd8674af6c0a` [29]; soroban-sim `d02938430f0ac41f34e5e3e4c8c48d85b871bd68` (soroban-env-host 29.0.2). OpenZeppelin stellar-contracts v0.8.0 (ddc327e); oz-policy-builder `43b13ed1d9c6662afd74f446006e69bfcf21259b`.

The simulation. SHA-256 of the page, `156f9875cdde076ddf728bc43e5caff9650179dcc296199362ed246c86c453e0`; of the native reference run, `bde6a1e79f37e69f34ae3bafad63fa606c64680399e2739b5ac922f97d62ce97`; and of the reference replies minus wall-clock timings, which the native run, the WASI build and the browser share, `925d02cbea59b1fb0fbd17d462463605ce2f82bc5a0a4e907bbcebc2ff4f8372`. Contract Wasm, SHA-256:

- Perch account `cc56a2d36669dba65c9089ba9c133e284d4b2f9bff826e2d52876e66b5e14428`
- document compiler `e89d977ac2bebaf935720a7a3408f6cd57b5940a02b0476e7b8e071e224b0fe3`
- Ed25519 verifier `27a733ceb0bda62fc9f645f3a708de73afccfdde9d4b2ffb30f44a2b1b862e7c`

- OpenZeppelin account `8286422f205951f37f4249401b403471fc432340dde6925432d31bb8a90cce27`
- OpenZeppelin `spending_limit` `65e70aad310bcaf7025aedc97e70cde098e1dd6c22d5153879c9cc64ea7da8e2`
- `policy-interpreter` `6704996f626904a4e33b95bda17b73963fc9cf502ffdfdbe9d85b4b10288e0d4`

The Perch session also loads two contracts its account pins but never calls, Perch's earlier interpreter (`4191ce8271035711c7bf9822d7cb20fcd0f300bf772b8defcc2279992440da6f`) and spending limit (`ccaa8f0404279aab8005791bffcbc819d8325c7abc114366cb6ef3472643f59b`). The Perch account is built with its committed infrastructure pins rewritten to the compiler, interpreter and spending-limit builds of the same run, so its bytes differ from a committed-pin build and from the testnet preview's. The three OpenZeppelin-side hashes equal oz-policy-builder's testnet records; built with stellar-cli 27.0.0, the account and `spending_limit` match after `normalize-oz.py` sets the CLI version string in their metadata to 27.1.0's. The page carries its build sources, whose `BUILD.md` fetches the pinned trees by commit and rebuilds the contracts, the driver (native and `wasm32-wasip1`), the reference run and the page.

Proofs. rule-core's proofs reproduce with Charon 7b462d9, Aeneas ab34bd2 and Lean v4.35.0: `tools/extract.sh --check` regenerates the translation and fails on any difference, and `lake build` in `formal/` checks the theorems. The earlier Perch-only experiment and its full reproducibility record are in the vision companion.

B Formal Definitions and Two Theorems

These are the definitions that the theorems of §8 use, and the two of its five theorems that the body states in words only; the verification companion gives all nine definitions and thirteen theorems. A subscript T marks the Lean translation of a Rust function, and $[\![\cdot]\!]$ maps the translation's values to the model's (Definition 9 of the verification companion).

Definition 1 (Contexts and authorizations)**.**

$$c \ ::= \ \mathsf{call}(C, f, \vec{v}) \ \mid \ \mathsf{create}(w)$$
$$a = (\vec{c}, \vec{j}, S, \ell, g)$$

A context is one call or contract creation in an authorization entry's invocation tree. An authorization a carries the entry's contexts $\vec{c} = c_1, \ldots, c_n$ in preorder, root first; one rule index j_i per context; the claimed signers S; the ledger ℓ; and the account's grant generation g. Lean: Chain.Context; Authorization

Definition 2 (Rules and *allows*)**.**

$$\begin{aligned}
&\mathit{allows}(r, c, S) \iff \mathit{sc}_r(c) \wedge \bigwedge_{\varphi \in \Phi_r} \varphi(c) \\
&\qquad\qquad \wedge\ k_r \le |\Sigma_r \cap S| \\
&\mathit{allows}(R, c, S) \iff \exists r \in R.\ \mathit{allows}(r, c, S) \\
&R' \sqsubseteq R \iff \forall c, S.\ \mathit{allows}(R', c, S) \\
&\qquad\qquad \implies \mathit{allows}(R, c, S)
\end{aligned}$$

A rule r has a scope sc_r (one contract and a function list; false on creations), argument conditions Φ_r, signers Σ_r and a threshold k_r over distinct signers, and optionally a cap (λ_r, P_r), a last ledger u_r and a generation g_r. *allows* reads none of the last three; *checkAuth* checks them. A rule set allows what one of its rules allows; $R' \sqsubseteq R$ when R allows, under the same signers, everything R' allows.

Lean: Rule.allows; RuleSet.allows; RuleSet.narrower

Definition 3 (*checkAuth* and runs)**.**

$$\begin{aligned}
&\mathit{checkAuth}(R, \rho, (\vec{c}, \vec{j}, S, \ell, g)) = \rho' \iff \\
&\quad |\vec{j}| = |\vec{c}| \wedge |\rho| = |R| \\
&\quad \wedge\ \forall i.\ j_i < |R| \wedge \ell \le u_{r_i} \wedge g_{r_i} \in \{\bot, g\} \\
&\qquad\quad \wedge\ \mathit{allows}(r_i, c_i, S) \\
&\quad \wedge\ \forall s \in S.\ \exists i.\ s \in \Sigma_{r_i} \\
&\quad \wedge\ \rho' = \mathit{charge}(\cdots \mathit{charge}(\rho, c_1, j_1) \cdots, c_n, j_n)
\end{aligned}$$

Here $r_i = R_{j_i}$. $\mathit{charge}(\rho, c, j)$ leaves ρ unchanged when R_j has no cap; otherwise it reads the amount c carries, refusing a call that carries none, and applies Definition 4's check to rule j's record. Each charge must succeed. *checkAuth* returns the next records or the first refusal, and a refusal changes nothing. A run applies authorizations in order, keeping the records on a refused one.

Lean: RuleSet.checkAuth; RuleSet.checkAuth_eq_ok_iff; RuleSet.run; RuleSet.Accepts

Definition 4 (Window, check and *spent*)**.**

$$\begin{aligned}
&\mathit{inWindow}_P(L, t) \iff \max(0,\, L - P) < t \le L \\
&\mathit{check}_{\lambda,P}(h, \ell, x) = h' \ \wedge\ x > 0 \implies \\
&\qquad \sum_{k:\ \mathit{inWindow}_P(\ell,\, t_k)} y_k \ +\ x \ \le\ \lambda \\
&\qquad \wedge\ h' = ((t_k, y_k))_{k:\ \mathit{inWindow}_P(\ell,\, t_k)} \\
&\qquad\quad \cdot\ (\ell, x)
\end{aligned}$$

A record is a list $h = ((t_k, y_k))_{k=1}^{m}$ of positive charges in ledger order, its total their sum. The check of a charge x at ledger ℓ refuses $x < 0$; accepts $x = 0$,

leaving h; and otherwise refuses a ledger before the newest charge, an overflow, 1,000 entries already kept, or a window total above λ. On acceptance it keeps the entries in the window at ℓ and appends (ℓ, x). $\mathit{spent}_j(R, \rho, \vec{a})$ lists (ℓ_a, x) for each positive charge that the accepted authorizations of run $\vec{a}$ make to rule j, in order. Lean: Params.inWindow; Params.check; RuleSet.spent

Definition 5 (Derivation)**.**

$$\begin{aligned} &\mathit{Derives}(R, R) \\ &\mathit{Derives}(R, R') \wedge \mathit{attenuate}(R', \sigma) = R'' \\ &\qquad \Longrightarrow\ \mathit{Derives}(R, R'') \end{aligned}$$

σ is one of rule-core's ten attenuation steps: drop a rule; remove functions, or restrict functions; add an argument condition; remove signers, or raise a threshold; add a cap, lower its limit, or lengthen its period; set or lower the last ledger. No step changes a rule's target or generation. $\mathit{attenuateAll}(R, \vec{\sigma}) = R'$ applies a list of steps and implies $\mathit{Derives}(R, R')$.

Lean: Chain.RuleSet.Derives; Chain.RuleSet.attenuate; Chain.Attenuation

Theorem 1 (Total equivalence of the evaluator).

$$\begin{aligned} \forall x.\ \exists y.\ &\mathit{checkAuth}_{\mathrm{T}}(x) = \mathsf{ok}(y) \\ \wedge\ &[\![y]\!] = \mathit{checkAuth}([\![x]\!]) \end{aligned}$$

Here x is the seven Rust inputs, and ok *is Aeneas's success, so neither a panic nor divergence. On every input the translated evaluator succeeds, never panicking or diverging, and its value denotes the model's result on the denoted inputs; this says nothing about allocation failure or compiled Wasm.*

Hypotheses: None: every input

Evidence: Equiv.check_auth_eq · PROVED · TRANSLATION

Theorem 4 (Attenuation narrows authority).

$$\mathit{Derives}(R, R') \ \Longrightarrow\ R' \sqsubseteq R$$

Successful attenuation by the ten attenuation steps yields a rule set narrower than its parent: under any signers, every call the child allows, the parent allows. Repeated derivation preserves this relationship. allows *reads no cap or expiry, so the steps that add or tighten them narrow only over runs (Theorem 5); parent and child keep separate records, and neither theorem bounds what they spend together. The account has no entry point that installs an attenuated grant at the pinned build.*

Hypotheses: Each derivation step is one of the ten attenuation steps and succeeds

Evidence: Chain.RuleSet.Derives.narrower; Extracted.Derives.narrower · PROVED · MODEL + TRANSLATION

References

1. Birgisson, A., Politz, J.G., Erlingsson, Ú., Taly, A., Vrable, M., Lentczner, M.: Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In: Network and Distributed System Security Symposium (NDSS) (2014), https://research.google/pubs/macaroons-cookies-with-contextual-caveats-for-decentralized-authorization-in-the-cloud/
2. Biscuit authorization token specification. biscuitsec.org, https://www.biscuitsec.org/
3. Buterin, V., Weiss, Y., Tirosh, D., Nacson, S., Forshtat, A., Gazso, K., Hess, T.: ERC-4337: Account abstraction using alt mempool. Ethereum Improvement Proposals (2021), https://eips.ethereum.org/EIPS/eip-4337
4. Certora: User guide for Sunbeam. Certora documentation, https://docs.certora.com/en/latest/docs/sunbeam/usage.html, contract specification and Wasm verification workflow
5. Cutler, J.W., Disselkoen, C., Eline, A., He, S., Headley, K., Hicks, M., Hietala, K., Ioannidis, E., Kastner, J., Mamat, A., McAdams, D., McCutchen, M., Rungta, N., Torlak, E., Wells, A.M.: Cedar: A new language for expressive, fast, safe, and analyzable authorization. Proceedings of the ACM on Programming Languages (OOPSLA) (2024), https://www.amazon.science/publications/cedar-a-new-language-for-expressive-fast-safe-and-analyzable-authorization
6. Disselkoen, C., Eline, A., He, S., Headley, K., Hicks, M., Hietala, K., Kastner, J., Mamat, A., McCutchen, M., Rungta, N., Shah, B., Torlak, E., Wells, A.: How we built Cedar: A verification-guided approach. In: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion) (2024), https://arxiv.org/abs/2407.01688
7. Ellison, C., Frantz, B., Lampson, B., Rivest, R., Thomas, B., Ylönen, T.: SPKI certificate theory. RFC 2693 (1999), https://www.rfc-editor.org/rfc/rfc2693
8. Gnosis Guild: Zodiac Roles Modifier: conditions. Zodiac documentation, https://docs.zodiac.eco/developers/roles/conditions
9. Hardy, N.: The confused deputy (or why capabilities might have been invented). ACM SIGOPS Operating Systems Review **22**(4) (1988), https://doi.org/10.1145/54289.871709
10. Ho, S., Boisseau, G., Franceschino, L., Prak, Y., Fromherz, A., Protzenko, J.: Charon: An analysis framework for Rust. arXiv:2410.18042 (2024), https://arxiv.org/abs/2410.18042
11. Ho, S., Protzenko, J.: Aeneas: Rust verification by functional translation. In: Proceedings of the ACM on Programming Languages (ICFP) (2022), https://arxiv.org/abs/2206.07185
12. Isailovic, L., Rein, D., Finlay, D., Chiang, D., Makarov, F., Gomes, P., Swenberg, C., Rosario, L., Bowman, I., Smale, J.: ERC-7715: Request permissions from wallets. Ethereum Improvement Proposals (2024), https://eips.ethereum.org/EIPS/eip-7715, draft
13. Kozhevin, D.: CAP-0046-11: Soroban authorization framework. Stellar Core Advancement Proposals, https://github.com/stellar/stellar-protocol/blob/master/core/cap-0046-11.md, final
14. Kozhevin, D.: CAP-0071: Authentication delegation and address-bound Soroban credentials. Stellar Core Advancement Proposals (2025), https://github.com/stellar/stellar-protocol/blob/master/core/cap-0071.md, final; protocol 27

15. McPeck, R., Finlay, D., Dawson, R., Chiang, D.: ERC-7710: Smart contract delegation. Ethereum Improvement Proposals (2024), https://eips.ethereum.org/EIPS/eip-7710, interface proposal (Draft)
16. MetaMask: Delegation Framework, revision b4b8c41. GitHub, MetaMask/delegation-framework (2026), https://github.com/MetaMask/delegation-framework/tree/b4b8c41fc912455a1af2300bfcd1276676b92a2e, delegations restricted by caveat enforcer contracts
17. de Moura, L., Ullrich, S.: The Lean 4 theorem prover and programming language. In: Automated Deduction – CADE 28, LNCS 12699. pp. 625–635 (2021), https://doi.org/10.1007/978-3-030-79876-5_37
18. Nido testnet preview, draft pull request nidohq/nido#356. GitHub, nidohq/nido (2026), https://github.com/nidohq/nido/pull/356
19. OASIS: eXtensible Access Control Markup Language (XACML) version 3.0. OASIS Standard (2013), https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html
20. OpenZeppelin: Stellar smart account contracts, v0.8.0 (ddc327e). GitHub, OpenZeppelin/stellar-contracts, packages/accounts (2026), https://github.com/OpenZeppelin/stellar-contracts/tree/ddc327eb92d63709436da3a8455f641c2d5476c5/packages/accounts/src, context rules, signers and policies
21. Perch deployment manifests, deployments/testnet.json and deployments/mainnet.json at 0b3752d. GitHub, bitswell-hq/stellar-registry___perch (2026), https://github.com/bitswell-hq/stellar-registry__perch/tree/0b3752d8a179a5952b05fbebd29131dbbf99fa80/deployments
22. Runtime Verification: Komet, revision a078e29. GitHub, runtimeverification/komet, https://github.com/runtimeverification/komet/tree/a078e296621a4f52fd1497fed64435e9948cc830, pinned contract-semantics source
23. Saltzer, J.H., Schroeder, M.D.: The protection of information in computer systems. Proceedings of the IEEE **63**(9) (1975), https://doi.org/10.1109/PROC.1975.9939
24. Schneider, F.B.: Enforceable security policies. ACM Transactions on Information and System Security **3**(1) (2000), https://doi.org/10.1145/353323.353382
25. soroban-sim, revision d029384. GitHub, theahaco/soroban-sim (2026), https://github.com/theahaco/soroban-sim/tree/d02938430f0ac41f34e5e3e4c8c48d85b871bd68, portable host and session driver; host 29.0.2
26. Stellar Development Foundation: Custom account example. Stellar developer documentation, https://developers.stellar.org/docs/build/smart-contracts/example-contracts/complex-account, soroban authorization payloads and contexts
27. UCAN specification. GitHub, ucan-wg/spec, https://github.com/ucan-wg/spec
28. Untangled Finance: OZ Policy Builder, revision 43b13ed (v5.2.0 line). GitHub, untangledfinance/oz-policy-builder (2026), https://github.com/untangledfinance/oz-policy-builder/tree/43b13ed1d9c6662afd74f446006e69bfcf21259b, policy synthesiser, policy-interpreter contract, execution adapter, custody gate and Prime seat
29. Wyndham, W.: soroban-rule-core, revision a3c92ba. GitHub, bitswell/soroban-rule-core (2026), https://github.com/bitswell/soroban-rule-core/tree/a3c92ba074dcac4052727b7599e5fd8674af6c0a, Rust evaluator, model, translation and proof trust inventory
30. Wyndham, W., Engelman, E.: Perch source, revision 0b3752d. GitHub, bitswell-hq/stellar-registry___perch (2026), https://github.com/bitswell-hq/stellar-registry__perch/tree/0b3752d8a179a5952b05fbebd29131dbbf99fa80, account, compiler, recovery, verifiers and canonical-document proof

31. Xu, Z., Stoller, S.D.: Mining attribute-based access control policies. IEEE Transactions on Dependable and Secure Computing **12**(5) (2015), https://doi.org/10.1109/TDSC.2014.2369048
32. zeroknots, Kopp, K., Lee, T., Makarov, F., Poon, E., Min, L.: ERC-7579: Minimal modular smart accounts. Ethereum Improvement Proposals (2023), https://eips.ethereum.org/EIPS/eip-7579, draft